WFH Attendance & Employee Monitoring System

Installation and Local Execution Guide



Dexa Group Software Engineer Technical Assessment

Author: **Abdurrahman Khairi**

Version 1.0, November 2026

© 2026 Abdurrahman Khairi. All Rights Reserved. Created strictly for Technical Assessment evaluation.

TABLE OF CONTENTS

TABLE OF CONTENTS 2
CHAPTER I SCOPE AND ARCHITECTURE 4
1.1 Local Assessment Scope 4
1.2 The Six Components and Their Ports 4
CHAPTER II PREREQUISITES AND A ZERO STATE CHECK 6
2.1 Step 1: Check Tool Versions 6
2.2 Step 2: Look at the project folder 7
2.3 Step 3: Make sure the ports are free 8
CHAPTER III ENVIRONMENT FILES 10
3.1 Copy the Templates 10
3.2 Environment Variable Reference 12
3.3 SMTP Email Options 15
3.4 Emails the System Sends 16
CHAPTER IV METHOD A, DOCKER COMPOSE (RECOMMENDED) 18
4.1 Before Start: The Face Model 18
4.2 Step 1: Build and Start All Containers 18
4.3 Step 2: Check the Containers and their Ports 19
4.4 Step 3: Watch the Face Model Load 20
4.5 Step 4: Database Tables and Seed Accounts 21
CHAPTER V METHOD B, MANUAL LOCAL INSTALLATION 23
5.1 Step 1: Start MySQL Only 23
5.2 Step 2: Python Environment and The Face Engine 24
5.3 Step 3: Install and Start the NestJS Backend 26
5.4 Step 4: Install and Start the React Frontend 27
CHAPTER VI CHECK THE INSTALLATION IN THE BROWSER 29

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

6.1 Step 1: Biometric Engine Swagger Page 29
6.2 Step 1.5: One to One Face Check Against the Engine 29
6.3 Step 2: API Gateway and The Microservice Links 31
6.4 Step 3: Real Email Delivery 32
6.5 Step 4: The Portal Sign In Page 32
6.6 Step 5: First sign in with a seeded account 32
6.7 Step 6: The Uploads Folder on Disk 34
CHAPTER VII TROUBLESHOOTING AND MAINTENANCE 35
7.1 The Camera Does Not Open 35
7.2 The Face Model is Slow or Missing 35
7.3 Duplicate Face Prevention and Sibling Exemption 35
7.4 Real Email Does Not Arrive 36
7.5 Face Enrollment Returns 409 Conflict 37
7.6 A Port is Already Taken 38
7.7 Useful reset and maintenance commands 38

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

CHAPTER I
SCOPE AND ARCHITECTURE

1.1 Local Assessment Scope

This project is built for local technical evaluation. You run everything on your own computer. The only outside account you need is a Gmail or Outlook mailbox, which the system uses to send its email.

- **Email** goes out through a real SMTP account, Google Gmail or Microsoft Outlook, set in the root .env. There is no local mail catcher. With SMTP_OVERRIDE_RECIPIENT set, every one time code, approval email and support ticket lands in your own inbox.
- **Photos and attachments** are written to the local disk under uploads/, sorted by year, department, role, person and month.
- **Face recognition** runs on your CPU with InsightFace and ONNX Runtime. The only download it needs is the buffalo_l model, once.

1.2 The Six Components and Their Ports

dexa group

#	Component	Technology	Port (host)	What it does
1	Frontend portal	React 18, Vite 5, Tailwind CSS. In Docker it is built and served by nginx.	5173 (container port 80)	The web portal you sign in to. nginx forwards /api and /uploads to the gateway.
2	API Gateway	NestJS 11 on Express, Node 22	3000	The only HTTP entry point. Checks tokens and roles, accepts photo uploads, reads EXIF data, and passes work to the two microservices over TCP. Swagger lives at /api/docs.
3	Employee microservice	NestJS 11 TCP microservice, TypeORM	3001 (TCP, not HTTP)	Sign in, tokens, one time codes, employee records, access requests, support tickets and the Database Studio. It also creates the tables and seeds the demo accounts.
4	Attendance microservice	NestJS 11 TCP microservice, TypeORM, ExcelJS	3002 (TCP, not HTTP)	Clock in and clock out, face match scoring, status rules, requests and approvals, dashboards, exports and the company WFH policy.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

5	Biometric engine	Python 3.10, FastAPI, Uvicorn, InsightFace buffalo_l on ONNX Runtime (CPU)	8000	Turns a face photo into a 512 dimensional ArcFace vector. Swagger lives at /docs.
6	Database	MySQL 8.0	3306	Nine tables, kept in the Docker volume dexa_mysql_data.

Docker Compose starts them in a fixed order and waits for each health check: MySQL and the biometric engine first, then the employee service (which creates the tables), then the attendance service, then the gateway, and the frontend last.

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

CHAPTER II
PREREQUISITES AND A ZERO STATE CHECK

2.1 Step 1: Check Tool Versions

Open PowerShell (or any terminal) and check what you have. For the Docker method you only need Docker Desktop with Compose v2. For the manual method you also need Node.js and Python.

Tool	Needed for	Version to use	Version on the test machine
Docker Desktop (or Docker Engine)	Method A and the database in Method B	24 or newer, with WSL 2 on Windows	29.1.3
Docker Compose	Both methods	v2 plugin (docker compose, with a space)	v5.0.1
Node.js and npm	Method B	Node 22 LTS (the Dockerfiles use node:22-alpine)	Node 22.11.0, npm 10.9.0
Python	Method B	3.10 (the Dockerfile uses python:3.10-slim)	3.10.6

dexa group

```
node -v
npm -v
python --version
docker -v
docker compose version
```

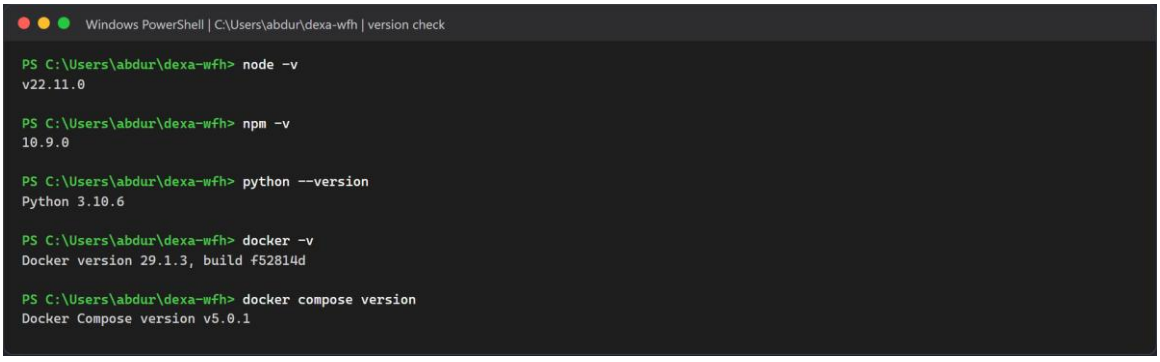


Figure 1.1: *Real version output on the test machine.*

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

You also want a webcam for the face steps, at least 8 GB of RAM (16 GB is more comfortable with six containers), and about 6 GB of free disk space for the images, the database and the face model. For email you need a Gmail account with two step verification and an App Password, or an Outlook account that allows SMTP sign in.

2.2 Step 2: Look at the project folder

The repository is a small monorepo. Each folder maps to one or more of the components above.

Folder	What is inside
backend/	One NestJS workspace with three apps (api-gateway, employee-service, attendance-service) and a shared library in libs/common for entities, DTOs, enums and rules. One Dockerfile builds all three.
biometric-service/	The FastAPI face engine: main.py, service.py, requirements.txt and its Dockerfile.
frontend/	The React app, its Vite config and the nginx config used in Docker.
database/	Optional SQL files. You do not need to run them, because the app creates its own tables.
uploads/	Where Docker mode stores photos. It starts empty apart from .gitkeep.
docker-compose.yml	Defines all six services, their ports, health checks and start order.

```
Get-ChildItem -Force -Name
```

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

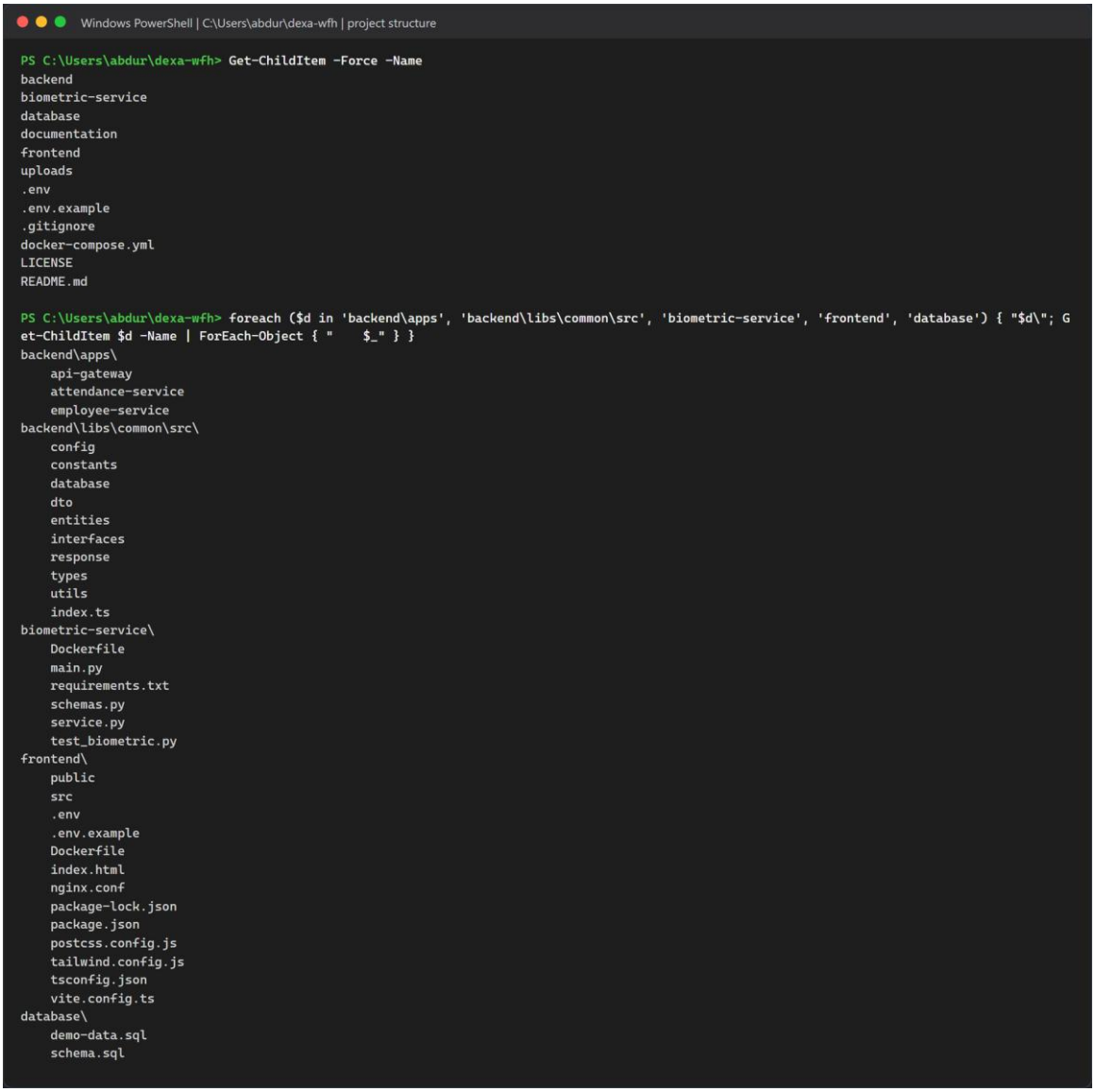


Figure 1.2: *The top level of the clean copy and the main subfolders. The three .env files in this listing were created in Chapter III.*

2.3 Step 3: Make sure the ports are free

The stack needs six ports. If another program already holds one of them, the matching container will fail to start. Run this check before you install anything:

```
foreach ($p in 3000,3001,3002,3306,5173,8000) {
  $c = Get-NetTCPConnection -State Listen -LocalPort $p -ErrorAction SilentlyContinue
  "{0,-6} {1}" -f $p, $(if ($c) { 'IN USE by PID ' + $c[0].OwningProcess } else { 'free' })
}
```

On macOS or Linux, `lsof -i :3306` does the same job for one port. On Windows, `netstat -ano | findstr :3306` works too.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

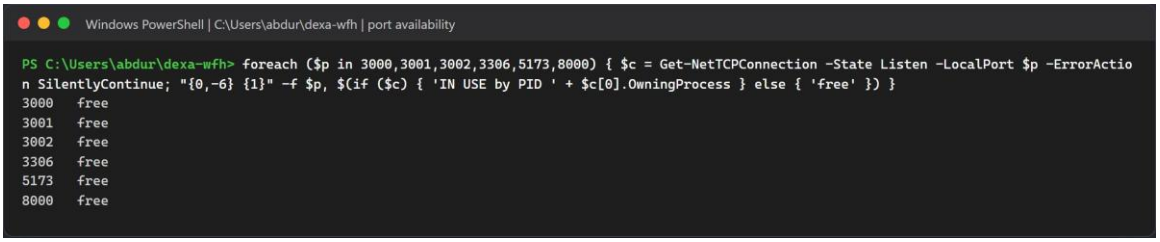


Figure 1.3: *All six ports reported free before the installation started.*

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

CHAPTER III
ENVIRONMENT FILES

3.1 Copy the Templates

The repository ships three templates. Copy each one to a real .env file:

```
Copy-Item .env.example .env
Copy-Item backend\.env.example backend\.env
Copy-Item frontend\.env.example frontend\.env
```

On macOS, Linux or Git Bash use cp .env.example .env and so on. The biometric service has no .env file at all. It only reads INSIGHTFACE_HOME and FORCE_GPU from the shell.

> **Warning: Fill in your SMTP account before the first start**
>
> The template ships with placeholders: SMTP_USER=your_email@gmail.com, SMTP_PASS=your_16_digit_app_password, and the same address in SMTP_FROM and SMTP_OVERRIDE_RECIPIENT. Replace them with your own values in the root .env before you run Docker. With the placeholders the stack still starts, but every email fails with an authentication error, and Employee Four cannot be activated because the activation code never arrives. Keep the single quotes around the SMTP_FROM value; without them Docker Compose stops with unexpected character "<" in variable name.

Here is what each file is for:

- **Root `.env`**: Docker Compose reads it to fill the ${VAR:-default} placeholders in docker-compose.yml. Every placeholder already has a default, so Docker also works with no .env at all. The NestJS apps read it too when you run them by hand.
- **`backend/.env`**: holds the same keys as the root file. The backend loads backend/.env first and then the root .env, so this copy is only useful for backend specific overrides.
- **`frontend/.env`**: holds VITE_API_BASE_URL=/api and an empty VITE_BACKEND_ORIGIN. Both are the defaults, so the file is optional.

These are the active lines of the root template, with its comment lines left out:

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

```
DB_TYPE=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USERNAME=dexa_user
DB_PASSWORD=dexa_password_2026
DB_DATABASE=dexa_wfh_db
DB_SYNCHRONIZE=true
DB_LOGGING=false
API_GATEWAY_PORT=3000
EMPLOYEE_SERVICE_HOST=127.0.0.1
EMPLOYEE_SERVICE_PORT=3001
ATTENDANCE_SERVICE_HOST=127.0.0.1
ATTENDANCE_SERVICE_PORT=3002
JWT_ACCESS_SECRET=dexa-wfh-jwt-access-secret-2026-abdurrahman-khairi
JWT_REFRESH_SECRET=dexa-wfh-jwt-refresh-secret-2026-abdurrahman-khairi
JWT_ACCESS_EXPIRES_IN=8h
JWT_REFRESH_EXPIRES_IN=7d
OTP_EXPIRES_MINUTES=10
WORK_START_TIME=08:30
LATE_GRACE_MINUTES=15
HALF_DAY_THRESHOLD_MINUTES=240
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_16_digit_app_password
SMTP_FROM='"Dexa Group WFH Monitoring" <your_email@gmail.com>'
SMTP_OVERRIDE_RECIPIENT=your_email@gmail.com
SYSTEM_NOREPLY_EMAIL=noreply-wfh@dexagroup.com
DEVELOPER_SUPPORT_EMAIL=abdurrahman.khairi@dexagroup.com
UPLOAD_DIR=./uploads
```


© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

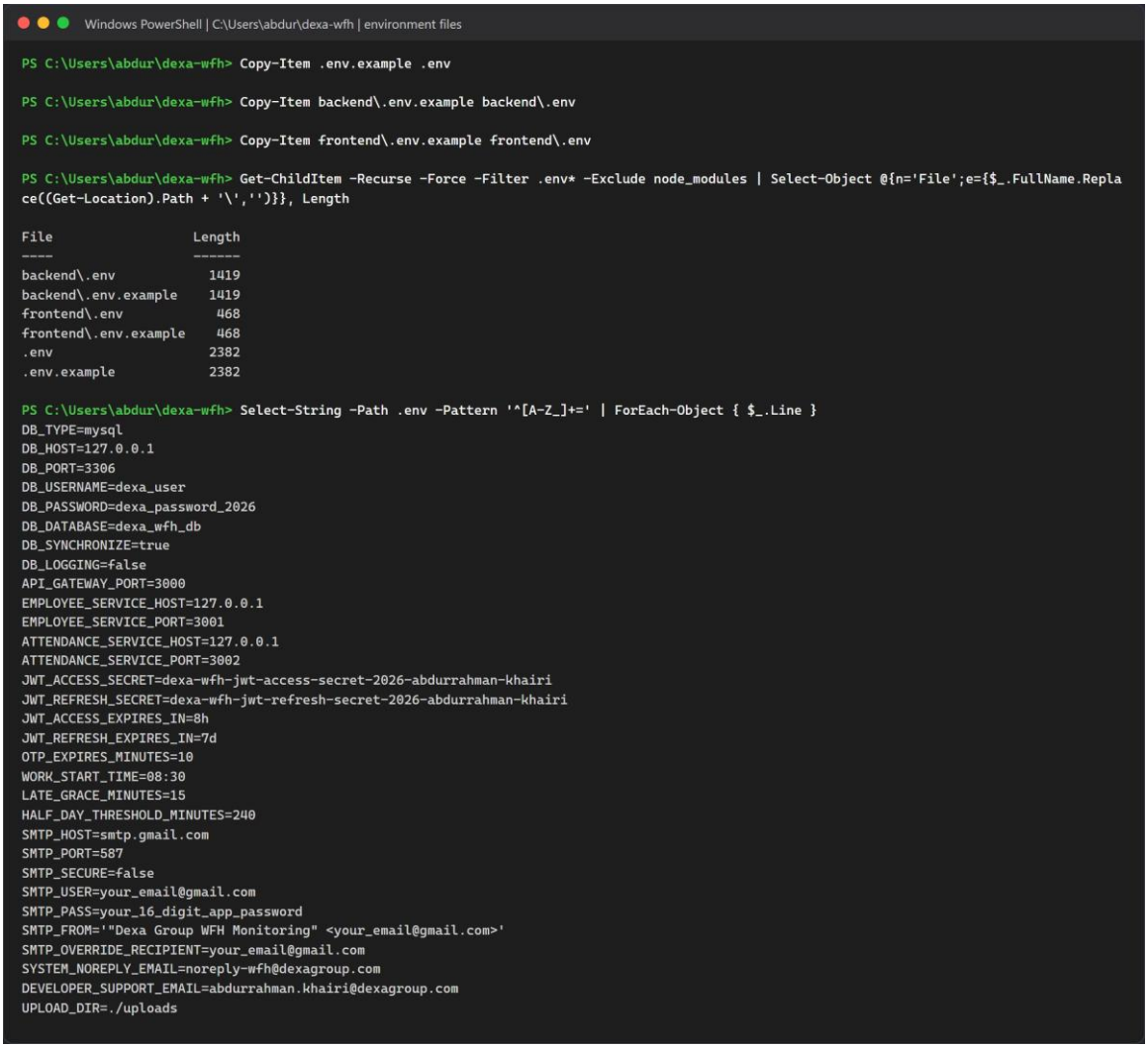


Figure 2.1: *The three .env files created from their templates, and the values inside the root .env.*

3.2 Environment Variable Reference

Inside Docker, the containers talk to each other by service name (mysql, biometric-service, employee-service, attendance-service). Those host names are written straight into docker-compose.yml and ignore your .env. When you run the apps by hand on your computer, everything is 127.0.0.1. The SMTP host is the exception: it is a real mail server on the internet in both methods.

Variable	Read by	Value in Docker	Value in Method B	What it controls
DB_HOST	employee and attendance services	mysql	127.0.0.1	Database host.
DB_PORT	employee and attendance services	3306 inside the network	3306	Database port. Setting it in the root .env also

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

				changes the host port Docker publishes.
DB_USERNAME, DB_PASSWORD, DB_DATABASE	both services and the MySQL container	dexa_user, dexa_password_2026, dexa_wfh_db	same	Credentials. The MySQL container creates this user and database on its first start.
DB_SYNCHRONIZE	employee service	true (attendance gets false)	true	Lets TypeORM create the tables. Only the employee service is allowed to.
EMPLOYEE_SERVICE_HOST and _PORT	gateway and employee service	employee-service, 3001	127.0.0.1, 3001	TCP link to the employee service.
ATTENDANCE_SERVICE_HOST and _PORT	gateway and attendance service	attendance-service, 3002	127.0.0.1, 3002	TCP link to the attendance service.
BIOMETRIC_SERVICE_URL	gateway and both services	http://biometric-service:8000	not set, so the code uses http://127.0.0.1:8000	Where the face engine lives.
JWT_SECRET	gateway and employee service	not set, so both use dexa-wfh-secret-2026	same	Secret that signs access tokens. The template calls it JWT_ACCESS_SECRET, but the code reads JWT_SECRET.
JWT_REFRESH_SECRET	employee service	template value	template value	Secret for refresh tokens.
JWT_ACCESS_EXPIRES_IN, JWT_REFRESH_EXPIRES_IN	employee service	8h, 7d	8h, 7d	Token lifetimes.
WORK_START_TIME, HALF_DAY_THRESHOLD_MINUTES	attendance service	08:30, 240	same	Shift start (WIB) and the minimum minutes for a full day.
WORK_GRACE_MINUTES	attendance service	not set, so 15	not set, so 15	Grace period before a clock in counts as late. The template calls it LATE_GRACE_MINUT

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

				ES, which the code does not read.
SMTP_HOST, SMTP_PORT	both services	smtp.gmail.com, 587	smtp.gmail.com, 587	The SMTP server. Gmail is the default; switch to smtp.office365.com for Outlook.
SMTP_SECURE, SMTP_USER, SMTP_PASS	both services	false, your address, your App Password	same, from backend/.env	TLS mode and SMTP sign in. Keep false for port 587; the connection then upgrades with STARTTLS. Port 465 switches to direct TLS on its own. Gmail and Outlook refuse mail without a sign in.
SMTP_FROM	both services	template value	template value	Sender address on every email. When it is empty, the sender becomes SMTP_USER, or the system address when that is empty too.
SMTP_OVERRIDE_RECIPIENT	employee and attendance services	your inbox	your inbox	When set, every email goes to this one address with a yellow banner naming the original recipient. The attendance service also drops the CC list; the employee service keeps it on support tickets.
OTP_EXPIRES_MINUTES	nothing	ignored	ignored	One time codes always expire after 15 minutes. That value is a constant in the code.
UPLOAD_DIR	nothing	ignored	ignored	Photos always go to uploads inside the

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

				folder the gateway runs from: /app/uploads in Docker (mapped to the root uploads/), backend/uploads in Method B.
VITE_API_BASE_URL	frontend build	/api	/api	Base URL for API calls. There is no VITE_API_URL variable in this project.
VITE_API_PROXY_TARGET	Vite dev server	not used	optional, default http://localhost:3000	Where npm run dev forwards /api and /uploads.
NODE_ENV	employee service	production	not set	When it is not production, API responses include the one time code as a hint, which helps local testing.

dexa group

3.3 SMTP Email Options

Section 5 of the root .env.example holds two blocks. Option A, Gmail, is active. To use Outlook, comment out the Option A lines and remove the # in front of the six lines of Option B. Fill in your own address and password either way, and keep only one block active.

```
# 5. SMTP Email Configuration (Real-World Gmail & Outlook Auto-Emailing)
# Option A (Active): Google Gmail SMTP (STARTTLS on port 587, 16 character Google App Password)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_16_digit_app_password
SMTP_FROM='"Dexa Group WFH Monitoring" <your_email@gmail.com>'


# Option B: Microsoft Outlook / Office 365 SMTP (STARTTLS on port 587):
# SMTP_HOST=smtp.office365.com
# SMTP_PORT=587
```


© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

```
# SMTP_SECURE=false
# SMTP_USER=your_email@outlook.com
# SMTP_PASS=your_outlook_password
# SMTP_FROM='"Dexa Group WFH Monitoring" <your_email@outlook.com>'


# Smart Evaluation Forwarding (delivers every email for the @dexagroup.com demo accounts to this inbox):
SMTP_OVERRIDE_RECIPIENT=your_email@gmail.com
```

- **Option A, Gmail.** SMTP_PASS must be a 16 character Google App Password, not your normal Google password. Chapter VII explains how to create one. Gmail sends as the account in SMTP_USER and replaces any other sender address unless it is a verified alias, so use that same Gmail address in SMTP_FROM.
- **Option B, Outlook and Office 365.** Use your full Outlook address as SMTP_USER. When the host contains office365.com or outlook.com, both services force STARTTLS and apply a cipher setting for Microsoft servers. Your account or tenant must still allow SMTP sign in with a password (see Chapter VII).
- **SMTP_FROM.** The sender shown on every email. Use the address you sign in with, and keep the single quotes around the whole value.
- **SMTP_OVERRIDE_RECIPIENT.** Your own inbox for evaluation. Leave it set while you test: the demo accounts use @dexagroup.com addresses that cannot receive real mail, so without it the activation codes and approval emails are lost. Method B reads backend/.env first, so put the same SMTP values there too.

Note: Recreate the containers after you edit .env

Docker hands environment values to a container only when it creates it, so docker compose restart keeps the old SMTP settings. Run docker compose up -d instead, which recreates every container whose settings changed. In Method B, stop npm run start:dev with Ctrl+C and start it again.

3.4 Emails the System Sends

Every email below goes out through the SMTP account in .env. With SMTP_OVERRIDE_RECIPIENT set, the To address of each one becomes your own inbox.

Email	Sent by	Who receives it, and when
Account activation code (6 digits, valid 15 minutes)	employee service	The account's own address, when an account that is not yet verified signs in, or when HR resends the code.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

Password reset code	employee service	The account's own address, from Forgot Password.
WFH, overtime, leave, sick or correction request	attendance service	To the department manager, CC the department HR contact (admin.hrd@dexagroup.com by default) and the requester. A request from a Manager or System Admin goes up to the HR contact instead, with sysadmin@dexagroup.com added to CC. Under the NOTICE_ONLY policy a WFH request is approved at once and the email is a notice. The review button opens /manager/approvals?requestId=<id>, and the reviewer still has to sign in.
Approval decision	attendance service	To the requester, CC the manager and admin.hrd@dexagroup.com, when a request is approved or rejected.
Support ticket (Contact Developer)	employee service	To every active System Admin, CC every active HR Admin and the sender.
SMTP test email	employee service	The targetEmail address, or the caller's own address. Sent by POST /api/support/test-email, for HR Admin and System Admin only.

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

CHAPTER IV
METHOD A, DOCKER COMPOSE (RECOMMENDED)

This is the quickest way to get the whole system running. One command builds five images, pulls MySQL, and starts all six containers in the right order.

4.1 Before Start: The Face Model

The biometric container does not download its model. It mounts your own model folder, %USERPROFILE%\.insightface on Windows or ~/.insightface on macOS and Linux, as read only. Make sure models/buffalo_l with its five .onnx files exists there before the first start. The simplest way is to download the release zip and unpack it:

```
New-Item -ItemType Directory -Force "$env:USERPROFILE\.insightface\models\buffalo_l" | Out-Null
Invoke-WebRequest -Uri "https://github.com/deepinsight/insightface/releases/download/v0.7/buffalo_l.zip" `
  -OutFile "$env:USERPROFILE\.insightface\models\buffalo_l.zip"
Expand-Archive "$env:USERPROFILE\.insightface\models\buffalo_l.zip" `
  -DestinationPath "$env:USERPROFILE\.insightface\models\buffalo_l" -Force
```

dexa group

If unpacking creates a nested buffalo_l\buffalo_l folder, move the five files up one level.

4.2 Step 1: Build and Start All Containers

```
docker compose up --build -d
```

Here is what gets built:

- **`biometric-service`**: Python 3.10 slim plus build tools, OpenCV, ONNX Runtime and InsightFace.
- **`employee-service`, `attendance-service`, `api-gateway`**: three images from the same backend Dockerfile. Each runs npm install and npm run build on Node 22.
- **`frontend`**: a Node 22 build stage that runs vite build, then an nginx image that serves the result.

MySQL is pulled as a ready made image. With -d, the command returns once the frontend container has started. Before that, it waits for each health check in turn, and you can watch the Healthy lines appear in that order.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

On the test machine the command took about three and a half minutes. Docker reused the cached npm install layers from an earlier build, but it ran pip install for the biometric image again (about two and a half minutes of that time) and rebuilt the application code. On a computer that has never built these images, expect it to take longer, because every package has to be downloaded first.

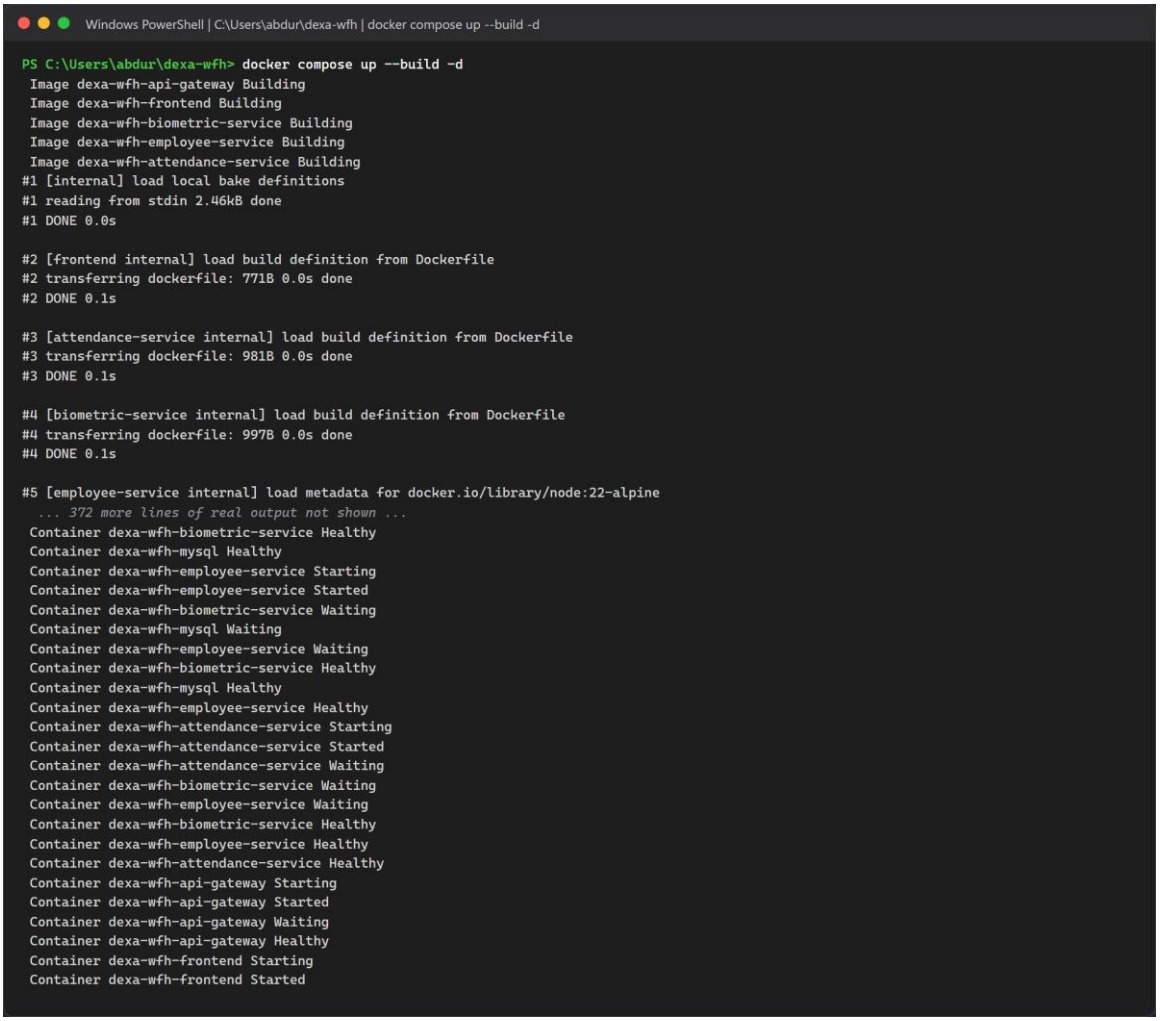


Figure 3.1: *The start and the end of the real build output, ending with every container started.*

4.3 Step 2: Check the Containers and their Ports

```
docker compose ps
```

Five containers should report healthy. The frontend has no health check, so it only shows Up. The ports column should match the table in Chapter I.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

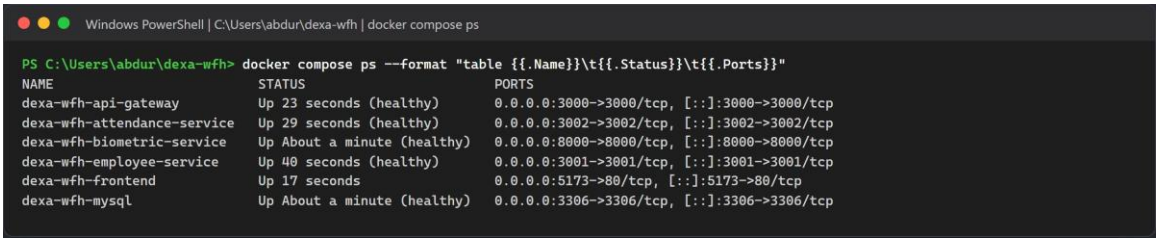


Figure 3.2: *All six containers running, with their host port bindings.*

4.4 Step 3: Watch the Face Model Load

When the biometric container starts, it loads the five buffalo_l ONNX models into memory before it answers any request. On the test machine this took about 28 seconds inside Docker. ArcFace (w600k_r50.onnx, a ResNet 50 network) is the model that produces the 512 dimensional face vectors. ONNX Runtime runs it on the CPU.

To prove the engine works end to end, send it one sample photo and read the log:

```
curl.exe -s http://localhost:8000/health
curl.exe -s -o NUL -w "HTTP %{http_code}\n" -F "file=@sample-face.jpg" http://localhost:8000/extract-embedding
docker compose logs biometric-service
```

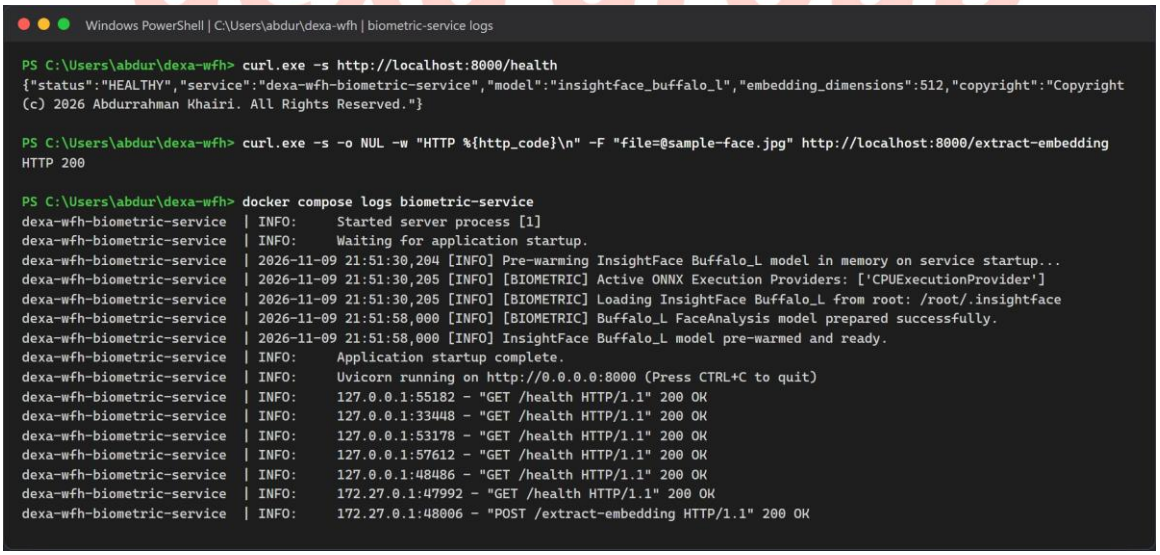


Figure 3.3: *The health response, a successful embedding request, and the log lines showing the CPU provider, the model folder and Uvicorn on port 8000.*

Note: If the model folder is missing

The container still reports healthy, but every face enrollment and clock in will fail. Check that models/buffalo_l exists in your home folder and run docker compose restart biometric-service.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

4.5 Step 4: Database Tables and Seed Accounts

There is no migration or seed command to type. When the employee service starts, TypeORM creates all nine tables, and the seeder fills an empty database with five departments and six accounts.

The two manager accounts are only added on the next start of the employee service. Run this once after the first boot:

```
docker compose logs employee-service | findstr /i "seed provision already listening"
docker compose restart employee-service
docker compose logs --since 1m employee-service | findstr /i "provision already listening"
```

On macOS or Linux, use grep -iE in place of findstr /i. Then list the accounts straight from MySQL:

```
docker compose exec mysql mysql -udexa_user -pdexa_password_2026 dexa_wfh_db -e "SELECT u.email, u.role, u.is_verified, e.employee_code FROM users u LEFT JOIN employees e ON e.user_id = u.id ORDER BY u.role, u.email;"
```

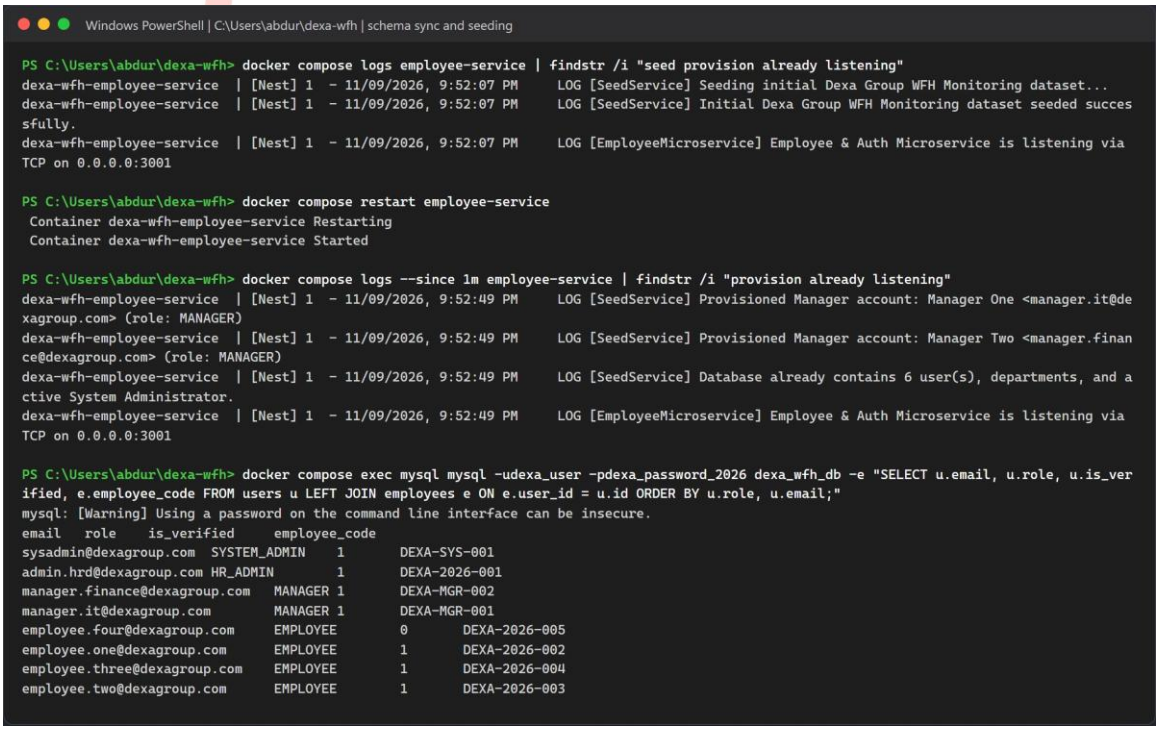


Figure 3.4: *First boot seeds the dataset, the restart adds Manager One and Manager Two, and MySQL lists all eight accounts.*

Warning: Skipping the restart

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

> Without the restart, manager.it@dexagroup.com and manager.finance@dexagroup.com do not exist. Employee requests still email the managers, but nobody can sign in as a manager to approve them.

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

CHAPTER V
METHOD B, MANUAL LOCAL INSTALLATION

Use this method when you want to change code and see it reload. MySQL still runs in Docker. The face engine, the three NestJS apps and the Vite dev server run directly on your computer. Each long running process gets its own terminal.

> **Note: Do not mix the two methods**
> If the Method A containers are still running, stop them first with docker compose down. Otherwise ports 3000, 3001, 3002, 5173 and 8000 are already taken.

5.1 Step 1: Start MySQL Only

```
docker compose down
docker compose up -d mysql
docker compose ps
docker compose exec mysql mysql -udexa_user -pdexa_password_2026 -e "SHOW DATABASES; SELECT VERSION();"
```

Name the service. A bare docker compose up -d would start the whole stack again. docker compose down without -v keeps the database volume, so the accounts seeded in Chapter IV are still there.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

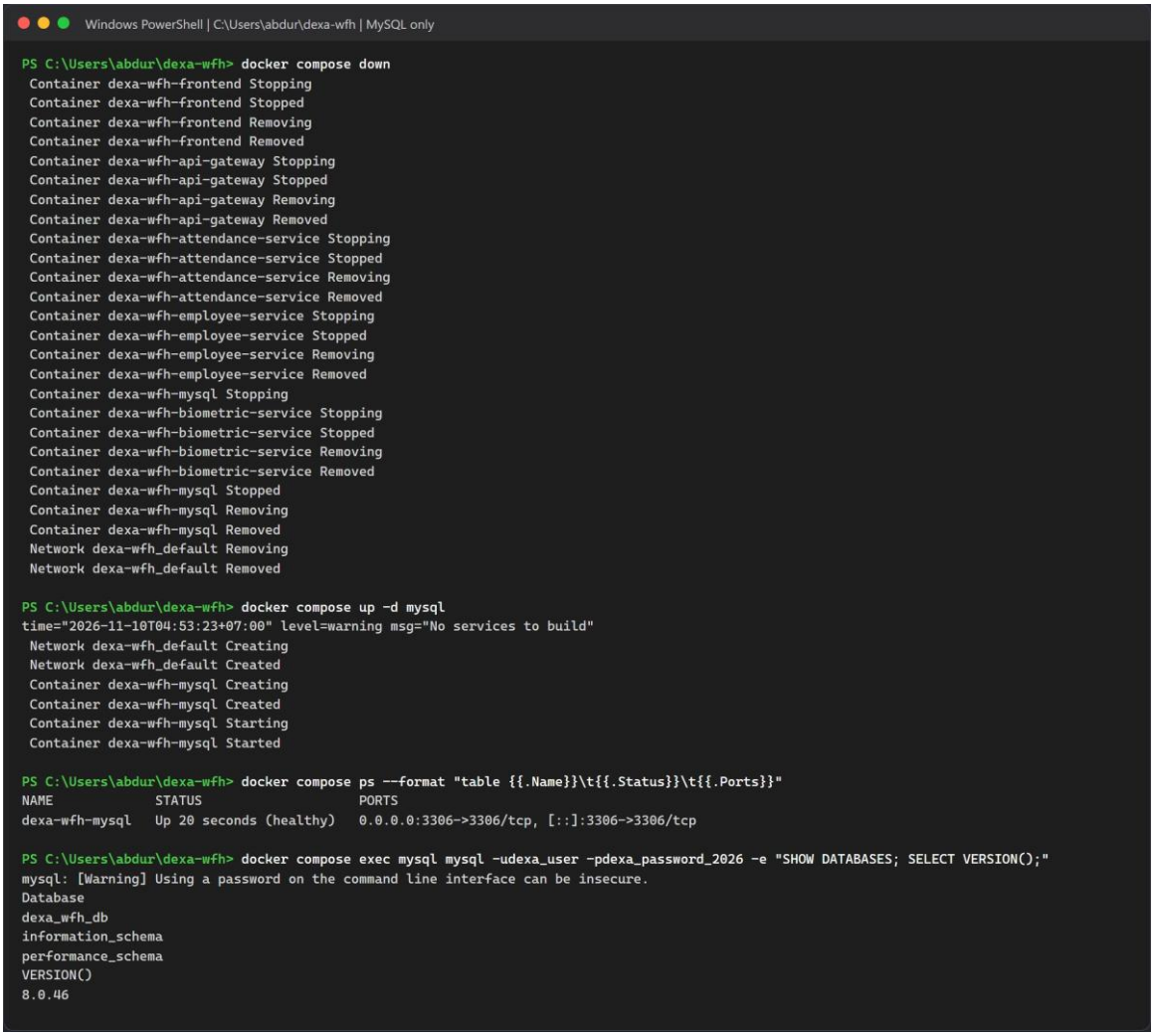


Figure 4.1: *MySQL 8.0 healthy and the dexa_wfh_db database present.*

5.2 Step 2: Python Environment and The Face Engine

In a new terminal:

```
cd biometric-service
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

On macOS or Linux, activate with source .venv/bin/activate. If PowerShell refuses to run the activation script, run Set-ExecutionPolicy -Scope CurrentUser RemoteSigned once.

On the test machine pip took a little under three minutes. It installed InsightFace 2.1, ONNX Runtime 1.23.2, NumPy 2.2.6 and OpenCV headless 5.0.0.93 as ready made wheels, so no C++ compiler was needed. The requirements file only sets minimum versions (>=), so pip picks the newest release on the day you install. Your versions may differ from these.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

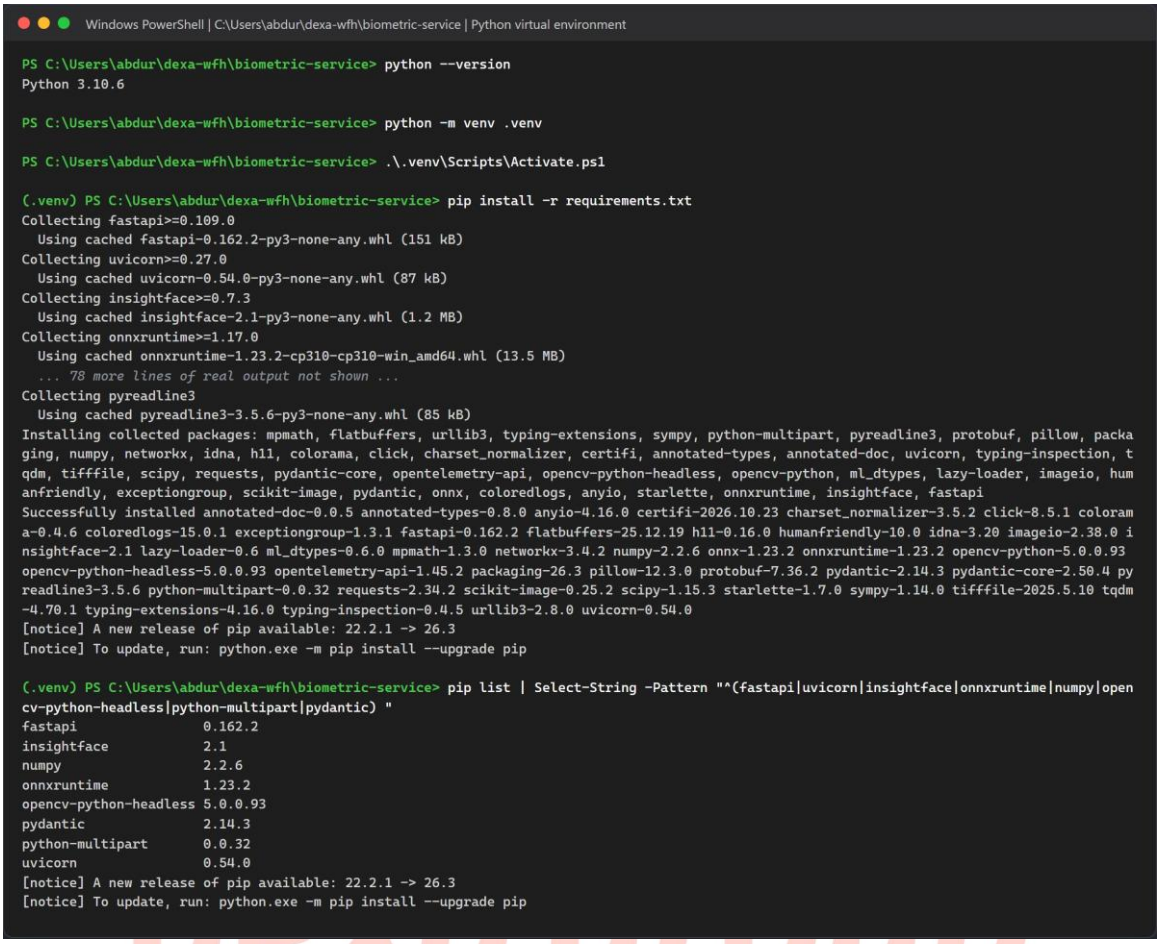


Figure 4.2: *Creating the virtual environment, the start and end of pip install, and the key packages it installed.*

Now start the engine:

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

It reads the model from your own ~/.insightface folder. If buffalo_l is missing there, InsightFace downloads it the first time it loads. python main.py also works if you do not want auto reload.

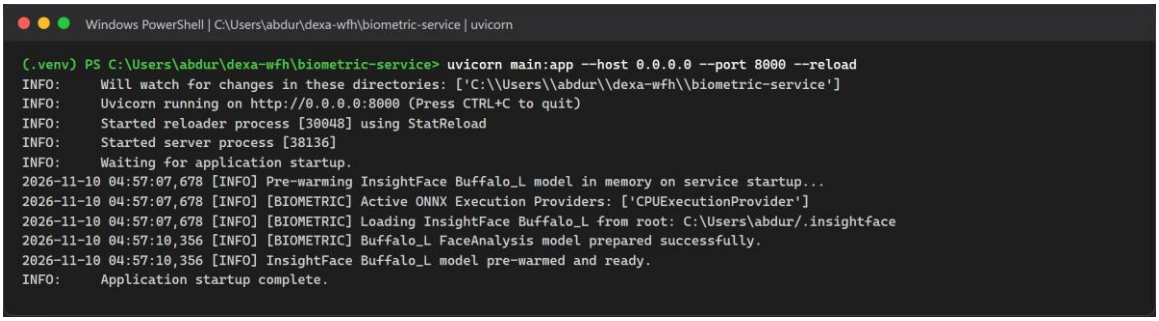


Figure 4.3: *Uvicorn listening on port 8000, with the model loaded from the local .insightface folder in under three seconds.*

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

5.3 Step 3: Install and Start the NestJS Backend

In another terminal:

```
cd backend
npm install
npm run start:dev
```

On the test machine npm install added 598 packages in 45 seconds. The deprecation and audit warnings come from third party packages and do not stop the install.

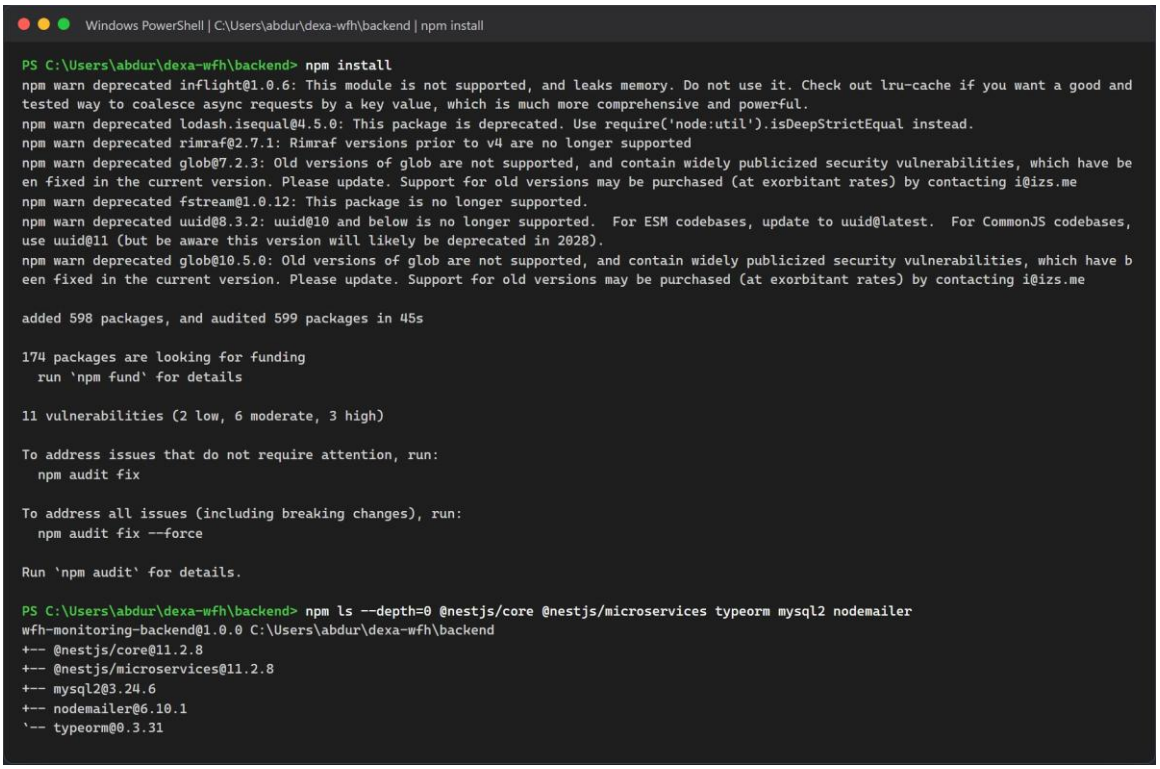


Figure 4.4: *npm install finished in the backend folder, with the main NestJS, TypeORM, MySQL and mail packages confirmed.*

npm run start:dev uses concurrently to run all three apps in watch mode. Each log line is tagged EMP, ATT or GW. There is no separate seed script here either. The employee service creates any missing tables and runs the seeder when it starts. On a fresh, empty database, start npm run start:employee first and let it finish before you start the other two.

In this mode the gateway saves uploaded photos to backend/uploads, because it uses the folder it was started from.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

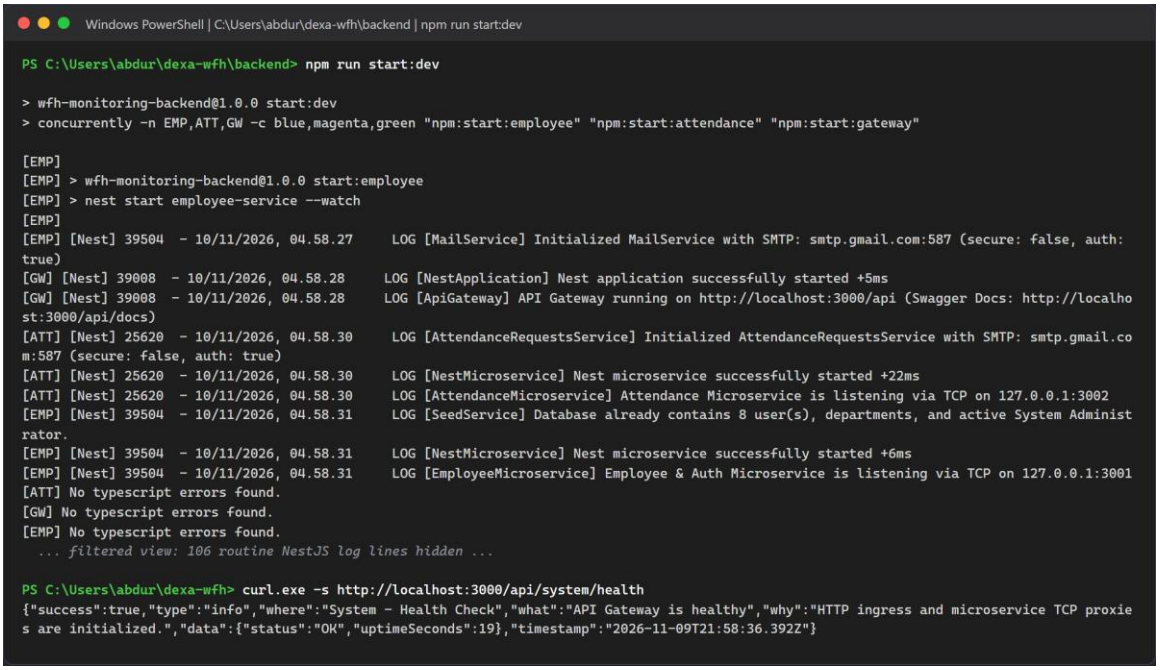


Figure 4.5: *The employee service on TCP 3001, the attendance service on TCP 3002 and the gateway on HTTP 3000, followed by a health check. Routine route mapping lines are hidden.*

5.4 Step 4: Install and Start the React Frontend

In a fourth terminal:

```
cd frontend
npm install
npm run dev
```

The dev server listens on port 5173 and forwards /api and /uploads to http://localhost:3000. You do not need a frontend .env for this.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

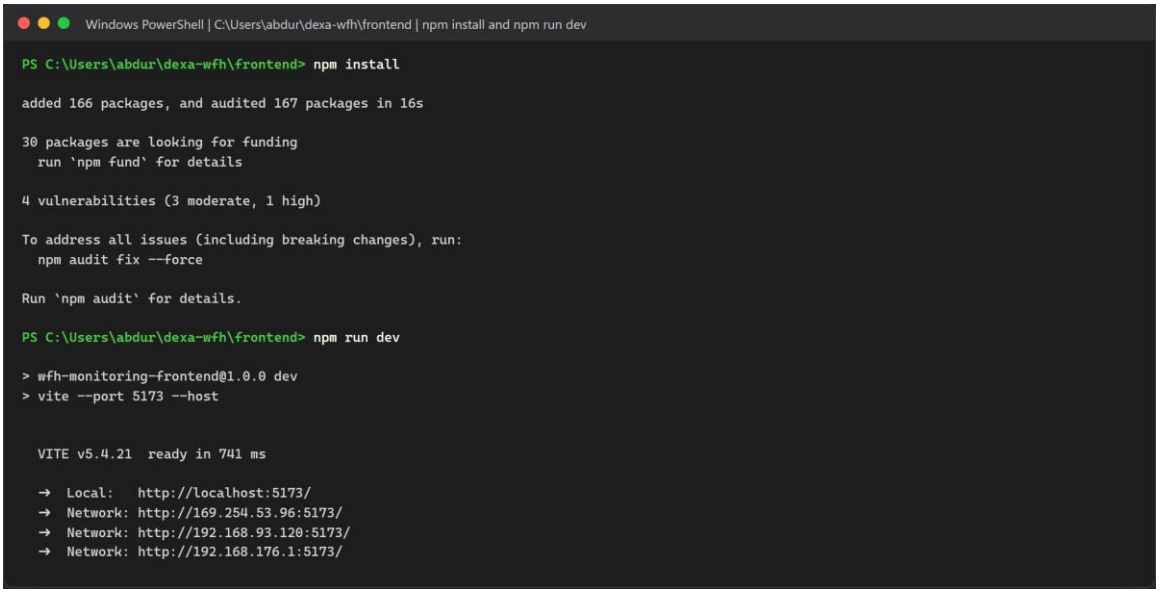


Figure 4.6: *npm install added 166 packages, and Vite reported ready at http://localhost:5173.*

Run the port check from Chapter II again. All six ports should now be in use. In this method that means node on 3000, 3001, 3002 and 5173, python on 8000, and Docker on 3306.

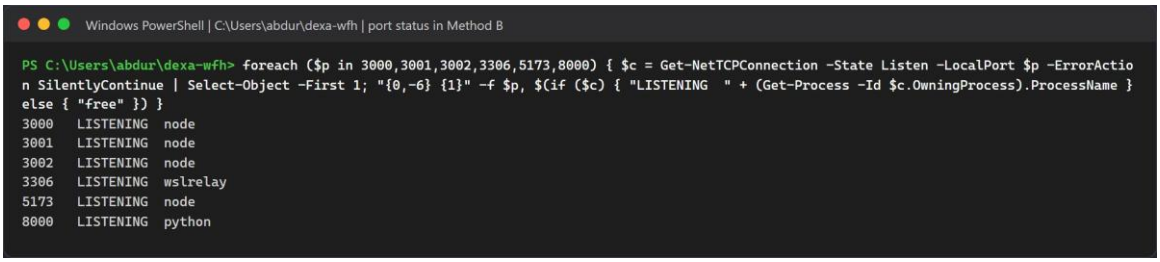


Figure 4.7: *Every port in use, with the process behind each one.*

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

CHAPTER VI
CHECK THE INSTALLATION IN THE BROWSER

The screenshots in this part were taken against the Docker stack from Chapter IV. With Method B the same addresses give the same pages.

6.1 Step 1: Biometric Engine Swagger Page

Open http://localhost:8000/docs. You should see three endpoints: GET /health, POST /extract-embedding and POST /verify-face. The backend only calls the first two. It compares faces itself, in the attendance service, and accepts a match at a similarity of 0.45 or higher.

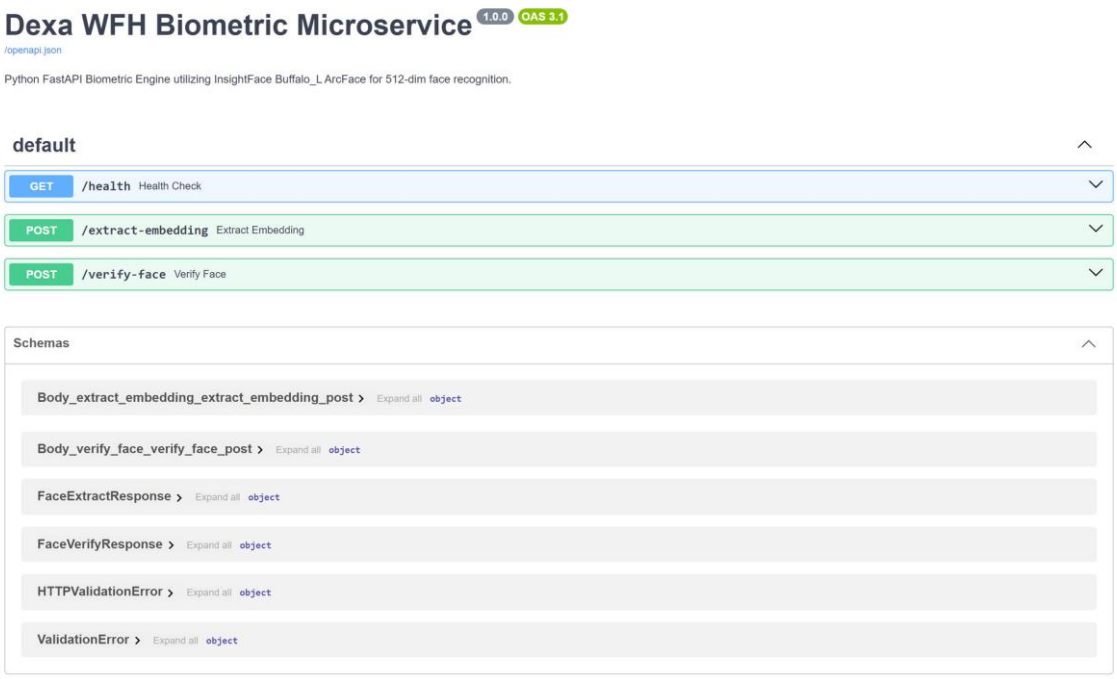


Figure 5.1: *The FastAPI Swagger page of the biometric engine.*

6.2 Step 1.5: One to One Face Check Against the Engine

This check talks to the face engine directly and writes nothing to the database. Put three photos in an empty folder: two of the same person and one of somebody else. The first call turns a photo into a 512 dimension ArcFace embedding. The next calls compare a new photo with that embedding by cosine similarity, which is the same check the attendance service runs at clock in and clock out.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

```
curl.exe -s -F "file=@person_a_1.jpg" http://localhost:8000/extract-embedding -o ref.json
$ref = (Get-Content ref.json | ConvertFrom-Json).embedding
Set-Content -Encoding ascii emb.json (ConvertTo-Json -InputObject $ref -Compress)
curl.exe -s -F "file=@person_a_2.jpg" -F "reference_embedding=<emb.json" http://localhost:8000/verify-face
curl.exe -s -F "file=@person_b.jpg" -F "reference_embedding=<emb.json" http://localhost:8000/verify-face
```

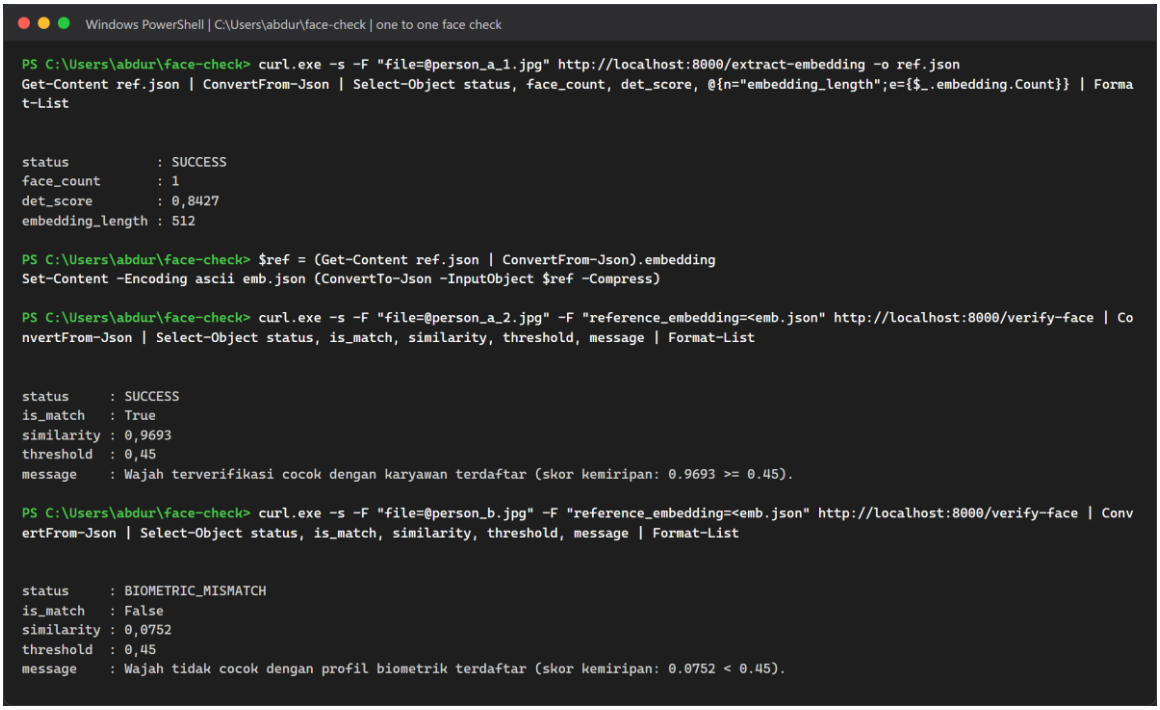


Figure 5.1b: *The same person scores 0.9693 and passes the 0.45 threshold. A different person scores 0.0752 and gets BIOMETRIC_MISMATCH. PowerShell prints decimals with a comma on this machine.*

These are the numbers the system works with. The 0.55 value sometimes quoted for face matching is the detection threshold, not the match threshold:

Check	Value	Where it lives
Face detection confidence	0.55, on a 320 x 320 detector input	det_thresh in biometric-service/service.py. A face below this score is not detected at all.
Clock in and clock out match (one to one)	similarity of 0.45 or more	attendance service, and the default threshold of POST /verify-face.
Duplicate check at enrollment (one to many)	below 0.50 unique; 0.50 up to 0.90 sibling tier; 0.90 or more identical	employee service, checked against every other active employee.
Live webcam photo time	within 5 minutes of server time	verifyPhotoTimestamp in libs/common attendance-rules.ts

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

Uploaded photo time	same WIB day, no older than 12 hours	same function. A photo outside the window is saved but marked as not verified.

Photo proof works like this. The portal draws every capture or upload onto an HTML5 canvas and burns in a watermark with the time, NIP, name, department and capture source, then sends that canvas JPEG. Because a canvas export carries no EXIF data, the gateway falls back to the capture time the browser sends in photoCapturedAt (for an upload, the file's last modified time). It only reads EXIF DateTimeOriginal, then CreateDate, then ModifyDate when an original JPEG reaches the API directly. The attendance service then marks the photo as verified or not verified with the rules in the table.

6.3 Step 2: API Gateway and The Microservice Links

Open http://localhost:3000/api/system/health. It returns a short JSON reply with "status":"OK".



Figure 5.2: *The gateway health reply in the browser.*

The health endpoint only proves that the gateway itself is up. It does not contact the microservices. To prove both TCP links work, call one endpoint that goes through each service:

- GET /api/system/policy reads the company policy through the attendance service on port 3002.
- POST /api/auth/login checks a password through the employee service on port 3001. HTTP 201 means the sign in worked.

```
curl.exe -s http://localhost:3000/api/system/policy
curl.exe -s -o NUL -w "HTTP %{http_code}" -X POST http://localhost:3000/api/auth/login -H "Content-Type: application/json" -d "{\"email\":\"sysadmin@dexagroup.com\",\"password\":\"AdminDexa2026!\"}"
```

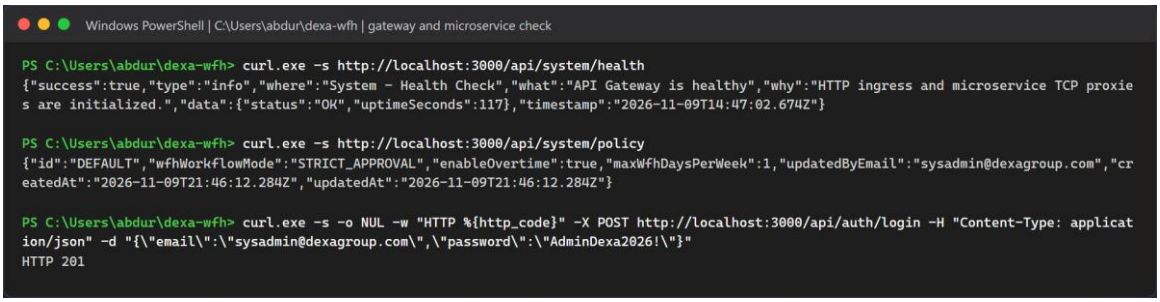


Figure 5.3: *The policy comes back from the attendance service and the login succeeds through the employee service.*

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

6.4 Step 3: Real Email Delivery

There is no local inbox to open. Every email goes through the SMTP account in .env, so open the mailbox you set in SMTP_OVERRIDE_RECIPIENT in Gmail or Outlook. Nothing arrives until the system sends something, so the next step sends a test email to prove the connection works. After that, one time codes, WFH notices, approval emails with their review links, and support tickets arrive in the same inbox, each with a yellow banner naming the demo account it was meant for.

6.5 Step 4: The Portal Sign In Page

Open http://localhost:5173. You land on the Dexa Group sign in page.



Figure 5.4: *The WFH Attendance Portal sign in page.*

6.6 Step 5: First sign in with a seeded account

These accounts come straight from the seeder in backend/apps/employee-service/src/seed/seed.service.ts:

Role	Full name	NIP	Department	Email	Password
SYSTEM_ADMIN	Abdurrahman Khairi	DEXA-SYS-001	IT & Digital Innovation	sysadmin@dexagroup.com	AdminDexa2026!
HR_ADMIN	HR Admin One	DEXA-2026-001	HRD & People Operations	admin.hrd@dexagroup.com	AdminDexa2026!

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

MANAGER	Manager One	DEXA-MGR-001	IT & Digital Innovation	manager.it@dexagroup.com	ManagerDexa2026!
MANAGER	Manager Two	DEXA-MGR-002	Finance & Accounting	manager.finance@dexagroup.com	ManagerDexa2026!
EMPLOYEE	Employee One	DEXA-2026-002	IT & Digital Innovation	employee.one@dexagroup.com	EmployeeDexa2026!
EMPLOYEE	Employee Two	DEXA-2026-003	Finance & Accounting	employee.two@dexagroup.com	EmployeeDexa2026!
EMPLOYEE	Employee Three	DEXA-2026-004	Quality Assurance & R&D	employee.three@dexagroup.com	EmployeeDexa2026!
EMPLOYEE (not verified yet)	Employee Four	DEXA-2026-005	IT & Digital Innovation	employee.four@dexagroup.com	set during verification

- The manager accounts only exist after the restart in Chapter IV, Step 4.
- Employee Four signs in once, receives a fresh 6 digit code by email, and then chooses a password. The code is valid for 15 minutes. In Docker the code is not shown on screen, so the SMTP settings must work for this account.
- Every role except the System Admin is asked to register a face on the first sign in.

Sign in as the System Admin. No face step is needed for this account, and the portal opens straight on the Database Management page.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

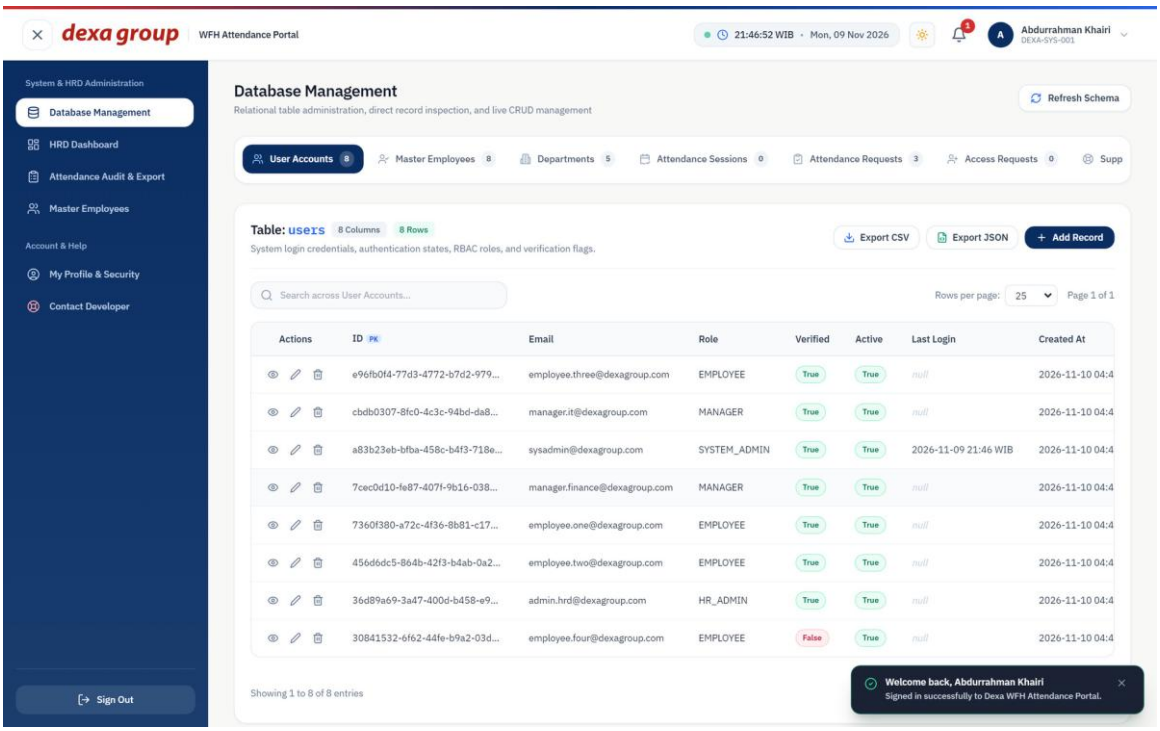


Figure 5.5: *Signed in as the System Admin right after installation. The users table holds the eight seeded accounts.*

6.7 Step 6: The Uploads Folder on Disk

Photos are stored in a tree: year, department, role folder, person, month. The role folder is manager for System Admins, HR Admins and managers, and employee for everyone else. File names carry a prefix, the person, the date and the time. The time in the file name is UTC, so it reads seven hours behind WIB.

To create the first file, the test run signed in as HR Admin One and registered a face. The gateway saved the enrollment photo straight away:

```
tree /F /A uploads
```



Figure 5.6: *The uploads tree after the first face enrollment.*

On macOS or Linux, use find uploads -type f. In Method B, look in backend/uploads instead.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

CHAPTER VII
TROUBLESHOOTING AND MAINTENANCE

7.1 The Camera Does Not Open

- Use http://localhost:5173. A LAN address such as http://192.168.x.x:5173 is not treated as secure, and the browser will block the camera there.
- In Chrome or Edge, click the site settings icon left of the address, set Camera to Allow, and reload. You can also open chrome://settings/content/camera or edge://settings/content/camera.
- On Windows, open Settings, then Privacy & security, then Camera, and allow access for desktop apps and the browser.
- Close Teams, Zoom or OBS if they are holding the camera.
- No camera at all? Use the photo upload option on the attendance page and give a reason. The upload is still accepted when its time is old, but it is marked as not verified unless it was taken on the same WIB day and no more than 12 hours ago. The face must still match your enrolled face.

7.2 The Face Model is Slow or Missing

- The model lives in %USERPROFILE%\.insightface\models\buffalo_l on Windows and ~/.insightface/models/buffalo_l on macOS and Linux. The zip is about 288 MB.
- If the download stalls, fetch https://github.com/deepinsight/insightface/releases/download/v0.7/buffalo_l.zip from another network or with a download manager, unpack it into that folder, and restart the engine.
- Docker mounts the folder as read only, so the container cannot download the model itself. Fix the folder on the host, then run docker compose restart biometric-service.
- The first load takes some time: about 28 seconds in Docker on the test machine. Give it a moment before the first face enrollment.

7.3 Duplicate Face Prevention and Sibling Exemption

This check needs a webcam and one real person. You enroll the same face on two accounts, watch the second attempt get refused, and then let HR allow it. Do it in this order:

- **Enroll Employee One.** Sign in as employee.one@dexagroup.com. The portal sends you to /auth/enroll-face. Capture your face and wait until the attendance page opens.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

- **Switch to Employee Four.** Sign out and sign in as employee.four@dexagroup.com. This account is not activated yet, so it first asks for the 6 digit code sent by email (to your own inbox when SMTP_OVERRIDE_RECIPIENT is set) and a new password (see Step 5), and then opens the face page.
- **Capture the same face.** The system refuses the enrollment with HTTP 409 Conflict and shows this message, where X is the similarity score: Wajah sudah terdaftar! Wajah ini memiliki kemiripan biometrik X% dengan profil karyawan 'Employee One' (DEXA-2026-002). Jika Anda memiliki saudara kandung atau kembar di Dexa Group, hubungi HR Admin untuk mengaktifkan Sibling Biometric Exemption.
- **Turn on the exemption.** Sign in as admin.hrd@dexagroup.com, open Employee Management (/hrd/employees), edit Employee Four and switch Sibling Biometric Exemption on. The page calls PATCH /api/employees/:id/sibling-exemption, which only HR Admin and System Admin accounts may call.
- **Enroll again.** Sign in as employee.four@dexagroup.com and capture the face again. This time enrollment succeeds, and the employee service writes a SIBLING_EXEMPTION_APPLIED line to its log with the score and the name of the admin who approved it.

  To see the result in the database:

```
docker compose exec mysql mysql -udexa_user -pdexa_password_2026 dexa_wfh_db -e "SELECT employee_code, full_name, is_sibling_exempt, face_embedding IS NOT NULL AS enrolled FROM employees WHERE email IN ('employee.one@dexagroup.com', 'employee.four@dexagroup.com');"
```

> **Note: A score of 90% or more is a different case**
>
> From 90% up, the message starts with "Wajah terdeteksi identik" instead, and the exemption does not help, because the system treats the capture as the same face registered twice. Whether two live captures of one person land below 90% depends on light, distance and pose. If you hit this block during the test, change the angle or the lighting a little and capture again.

7.4 Real Email Does Not Arrive

Start with the logs. On startup each service prints the SMTP host, port and whether it signs in, and every email ends in an [SMTP_SUCCESS] or [SMTP_ERROR] line:

```
docker compose logs employee-service | findstr /i "SMTP"
docker compose logs attendance-service | findstr /i "SMTP"
```


© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

- **Gmail error 535 5.7.8 "Username and Password not accepted".** Gmail does not accept your normal Google password over SMTP. Turn on two step verification for the Google account, then open Google Account, Security, two step verification, and App passwords at the bottom of that page. Create an app password, paste the 16 characters into SMTP_PASS without the spaces, and run docker compose up -d.
- **Gmail shows a different sender.** Gmail rewrites the sender to the account in SMTP_USER unless the address is a verified alias. Set SMTP_FROM to your Gmail address.
- **Outlook or Office 365 fails during STARTTLS.** Keep SMTP_HOST=smtp.office365.com, SMTP_PORT=587 and SMTP_SECURE=false, so the connection starts in plain text and then upgrades with STARTTLS. With SMTP_SECURE=true on port 587 the handshake fails. For Microsoft hosts both services also force STARTTLS (requireTLS) and set the TLS cipher list to SSLv3. That is an OpenSSL cipher group kept for compatibility with Microsoft servers; it does not switch the connection to the old SSL 3.0 protocol.
- **Outlook refuses a correct password.** Microsoft has been turning off password sign in for SMTP on many accounts and tenants. This project signs in with a password only and has no OAuth2 support, so if your account or tenant has SMTP AUTH turned off, switch to Gmail with a 16 character App Password.
- **Bounce notices after a support ticket.** With SMTP_OVERRIDE_RECIPIENT set, the employee service changes the To address of a support ticket but keeps its CC list, which holds the HR Admin and sender addresses on @dexagroup.com. Real SMTP then reports those copies as undeliverable. The ticket itself still reaches your inbox.
- **Nothing changed after you edited .env.** docker compose restart keeps the old values. Run docker compose up -d so Docker recreates the containers with the new ones.

7.5 Face Enrollment Returns 409 Conflict

- **"Wajah sudah terdaftar!" with a score from 50% to 89.9%.** The new face looks like another active employee. If the two people really are siblings or twins, an HR Admin opens Employee Management, edits the employee who is enrolling and switches Sibling Biometric Exemption on. The employee then captures the face again.
- **"Wajah terdeteksi identik" with 90% or more.** The system treats this as one face on two accounts, and the exemption does not lift it. Only active employees take part in the comparison, so during testing you can deactivate the other account in Employee Management, or start again from an empty database with docker compose down -v.
- **A 409 on an account that never enrolled before.** Another active account already holds a similar face, often from an earlier test run. The SQL query in Chapter VI, Step 2.5 shows which accounts have a face on file.

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

- **The exemption switch is refused with 403.** Only HR Admin and System Admin accounts may change it. Sign in with one of those roles.

7.6 A Port is Already Taken

```
# Windows
netstat -ano | findstr :3306
taskkill /PID <pid> /F

# macOS and Linux
lsof -i :3306
kill -9 <pid>
```

Local MySQL installs from XAMPP or Laragon often sit on 3306. You can stop them, or set DB_PORT=3307 in the root .env so Docker publishes MySQL on 3307. The containers keep talking to each other on 3306. The other host ports are fixed in docker-compose.yml. To move one, change the number on the left of its ports entry.

7.7 Useful reset and maintenance commands

dexa group

Goal	Command
Stop everything and keep the data	docker compose stop
Remove the containers and keep the database	docker compose down
Wipe the database volume for a clean slate	docker compose down -v
Seed again from zero	docker compose down -v, then docker compose up -d, then docker compose restart employee-service once for the managers
Rebuild after code changes	docker compose up --build -d
Rebuild one service	docker compose up --build -d api-gateway
Follow the logs of one service	docker compose logs -f attendance-service
Open a MySQL shell	docker compose exec mysql mysql -udexa_user -pdexa_password_2026 dexa_wfh_db

Warning: Wiping the volume

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited

> docker compose down -v deletes every account change, attendance record, request and enrolled face for good. It does not delete the photos in uploads/. Remove those by hand if you want a truly clean start, and keep the .gitkeep file.

dexa group

© 2026 Abdurrahman Khairi - Created strictly for Technical Assessment evaluation. Unauthorized commercial use is prohibited